

6. **Version:** 1.0
 7. **Paquete base:** sga
 8. **Finalizar.**
-

3 Configurar el Proyecto

1. **Renombrar el Proyecto:**
 - Clic derecho en el proyecto → **Rename** → Escribir: sga-jee-web
 2. **Configurar JDK 21:**
 - Clic derecho en el proyecto → **Properties**.
 - Ir a **Build** → **Compile**.
 - En **Target Release**, seleccionar **JDK 21**.
 - Marcar **Compile on Save**.
 3. **Eliminar Código Innecesario:**
 - En **Source Packages**, eliminar cualquier código generado automáticamente.
 4. Mantener los archivos **pom.xml**, **web.xml**, **glassfish-web.xml** y **beans.xml** sin modificar.
-

4 Copiar Código del Proyecto Anterior

Del proyecto **sga-jee**, copiar las siguientes clases:

1. Clase **Persona.java**

```
package sga.domain;

import java.io.Serializable;

public class Persona implements Serializable{
    private static final long serialVersionUID = 1L;
    private int idPersona;
    private String nombre;
    private String apellido;
    private String email;
    private String telefono;

    public Persona() {
    }

    public Persona(int idPersona, String nombre,
        String apellido, String email, String telefono) {
        this.idPersona = idPersona;
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
    }
}
```

```
public int getIdPersona() {
    return idPersona;
}

public void setIdPersona(int idPersona) {
    this.idPersona = idPersona;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellido() {
    return apellido;
}

public void setApellido(String apellido) {
    this.apellido = apellido;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

@Override
public String toString() {
    return "Persona{" + "idPersona=" + idPersona + ", nombre="
        + nombre + ", apellido=" + apellido + ", email=" + email + ", telefono="
        + telefono + '}';
}
}
```

✦ Explicación:

- Es la clase **Entidad** del sistema.
- Representa a una persona con sus atributos básicos.

5 Crear la Interfaz Local del EJB

✦ PersonaService.java (Nueva interfaz local del EJB)

```
package sga.servicio;

import java.util.List;
import jakarta.ejb.Local;
import sga.domain.Persona;

@Local
public interface PersonaService {
    public List<Persona> listarPersonas();

    public Persona encontrarPersonaPorId(Persona persona);

    public Persona encontrarPersonaPorEmail(Persona persona);

    public void registrarPersona(Persona persona);

    public void modificarPersona(Persona persona);

    public void eliminarPersona(Persona persona);
}
```

✦ Explicación:

- **@Local:** Indica que este EJB solo se usará dentro del mismo servidor.
- Define los métodos que implementará el EJB.

6 Modificar el EJB para usar CDI

✦ PersonaServiceImpl.java

```
package sga.servicio;

import java.util.ArrayList;
import java.util.List;
import jakarta.ejb.Stateless;
import sga.domain.Persona;

@Stateless
public class PersonaServiceImpl implements PersonaServiceRemote, PersonaService{

    @Override
    public List<Persona> listarPersonas() {
        List<Persona> personas = new ArrayList<>();
        personas.add(new Persona(1,"Rafael","Morales","rmorales@mail.com","55669900"));
        personas.add(new Persona(2,"Martha","Suarez","msuarez@mail.com","55443322"));
        return personas;
    }

    @Override
    public Persona encontrarPersonaPorId(Persona persona) {
```

```
        return null;
    }

    @Override
    public Persona encontrarPersonaPorEmail(Persona persona) {
        return null;
    }

    @Override
    public void registrarPersona(Persona persona) {
    }

    @Override
    public void modificarPersona(Persona persona) {
    }

    @Override
    public void eliminarPersona(Persona persona) {
    }
}
```

✚ Explicación:

- **@Stateless:** EJB sin estado.
- Implementa tanto la interface remota como la interface local **PersonaService** para manejar las operaciones.

7 Crear el Servlet para Usar el EJB Local con CDI

✚ PersonaServlet.java

```
package sga.web;

import java.io.IOException;
import java.util.List;
import jakarta.inject.Inject;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import sga.domain.Persona;
import sga.servicio.PersonaService;

@WebServlet("/personas")
public class PersonaServlet extends HttpServlet{

    @Inject
    PersonaService personaService;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException{
```

```

        List<Persona> personas = personaService.listarPersonas();
        System.out.println("personas:" + personas);
        request.setAttribute("personas", personas);
        request.getRequestDispatcher("/listadoPersonas.jsp").forward(request, response);
    }
}

```

✦ Explicación:

- **@Inject:** Usa CDI para inyectar el EJB.
- **doGet:** Llama al EJB y envía datos a la vista.

8 Modificar `index.html` para agregar un enlace al Servlet

✦ Ubicación: Web Pages → `index.html`

✦ Código a agregar/modificar:

```

<!DOCTYPE html>
<html>
<head>
    <title>Listado de Personas</title>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
</head>
<body>
<a href="personas">Ir al listado de personas</a>
</body>
</html>

```

✦ Explicación:

- Este archivo es la página de inicio de nuestra aplicación web.
- Se agrega un enlace que redirige al servlet `PersonaServlet`, el cual se encuentra mapeado en: `@WebServlet("/personas")`
- Cuando el usuario haga clic en el enlace, se ejecutará el método `doGet()` del servlet `PersonaServlet`, que llamará al **EJB Local**, obtendrá la lista de personas y la enviará a la vista `listadoPersonas.jsp`

9 Crear la Vista JSP

✦ `listadoPersonas.jsp`

```

<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>

```

```
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>Listado de Personas</title>
</head>
<body>
<h1>Listado de Personas</h1>
<ul>
  <c:forEach items="${personas}" var="persona">
    <li>${persona.nombre} ${persona.apellido} </li>
  </c:forEach>
</ul>
</body>
</html>
```

✚ Explicación:

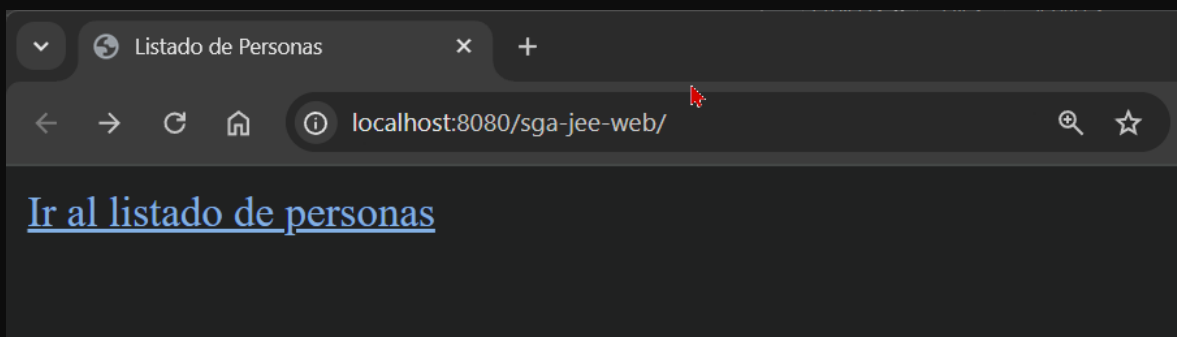
- **JSTL:** Recorre la lista de personas y la muestra en una lista HTML.

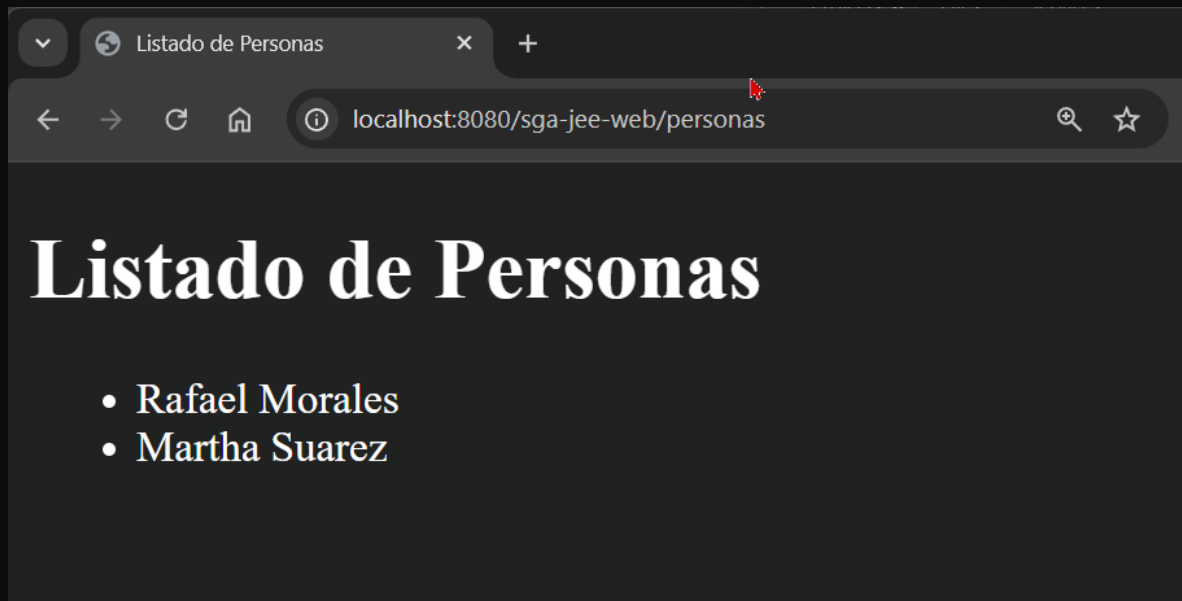
10 Desplegar y Ejecutar

1. **Despublicar sga-jee en GlassFish** (Para evitar conflictos con los EJB publicados anteriormente).
 1. Ir a la consola de Admin de Glassfish
 2. Ir a Applications
 3. Seleccionar sga-jee y dar click en undeploy
2. **Detener GlassFish.**
3. **Limpiar y Compilar el Proyecto** (Clean and Build).
4. **Ejecutar (Run).**

✅ Abrir en el navegador:

<http://localhost:8080/sga-jee-web>





🎉 ¡Listo! Aplicación SGA Web con CDI y EJB Local ya está funcionando! 🚀

El código fuente completo lo puedes descargar en la sección de recursos de este video.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)